

TOPIC 51/60

Types of RAG

Beyond Simple Lookups.

How RAG evolved from simple, linear keyword-matching pipelines to adaptive, self-correcting autonomous reasoning networks.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



TYPE 1

Naive RAG

The classical "Retrieve-and-Read" baseline. A linear, rigid pipeline with zero dynamic adjustments or quality evaluation loops.

Sequential Flow:

User Query → Vector DB Match → Stuff Prompt → LLM Response

Severe Production Vulnerabilities:

- **Low Precision:** Stuffing irrelevant context chunks (noise) into the prompt.
- **Vulnerable Retrieval:** Zero correction if the initial database search returns bad results.
- **Redundant Costs:** Exposes entire context length limits to static, duplicate documents.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



TYPE 2

Advanced RAG

Solving the Naive bottleneck. Advanced RAG introduces optimization layers **before** and **after** the vector database querying phases.

A. Pre-Retrieval Layer

- **Query Expansion:** Rewriting vague user prompts into rich vector terms.
- **Sub-Querying:** Breaking complex requests down into small separate lookups.

B. Post-Retrieval Layer

- **Re-ranking:** Re-ordering top vector hits with high-precision Cross-Encoders.
- **Context Squeezing:** Trimming noise out of extracted chunks to optimize token windows.

** Boosts target precision metrics from ~65% up to 85%+ in production enterprise tests.*

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



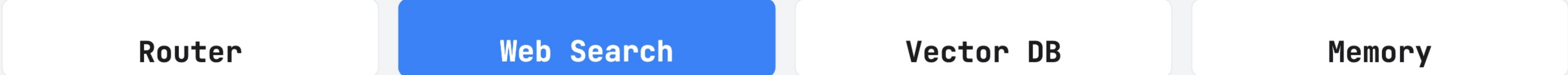
TYPE 3

ModuLar RAG

A highly dynamic, non-linear architecture. Instead of running a fixed conveyor belt, components are treated as **plug-and-play modules** controlled by an orchestrator router.

ModuLar Orchestration Loop

Depending on user intent, the router selects different specialized paths. It can query web engines, refer to an isolated long-term SQL transaction ledger, or trigger a local vector index on the fly.



TYPE 4

Corrective RAG

Corrective RAG (CRAG) evaluates the quality of retrieved contexts before passing them to the model, acting as an automated compliance auditor.

CORRECT

Proceed directly to generation phase with confidence.

INCORRECT

Discard completely. Route directly to Web Search fallbacks.

AMBIGUOUS

Blend raw database results with web searches for hybrid answers.

** Evaluates database relevance using light classifier models to avoid poison prompts.*

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



TYPE 5

Self-Reflective RAG

Self-RAG is a framework that trains a model to reflect on its own answers using specialized internal critique tokens.

- IsRetrieve

Do I actually need context for this step? [YES / NO]
- IsReL

Is the retrieved information relevant to the user query? [RELEVANT / IRRELEVANT]
- IsSup

Is the generated answer fully supported by facts in the context? [SUPPORTED / CONTRADICTED]
- IsUse

Is the output final answer useful? [HIGH / MEDIUM / LOW]



TYPE 6

Graph RAG

Standard RAG is blind to relations. **Graph RAG** extracts knowledge graphs (Entities, relationships, and claims) and maps them directly alongside vector indexes.

Unlocking Relational Intelligence:

- **Multi-Hop Queries:** Linking complex connection lines (e.g. "How is Vendor X related to security flaw Y?").
- **Global Summarization:** Analyzing huge volumes of data to discover thematic overviews instead of isolated factoids.
- **Entity Extraction:** Generating clean semantic nodes to capture structural connections.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



TYPE 7

Agentic RAG

RAG as an autonomous workspace tool. The LLM acts as an active **agent** within a loop, controlling when and what to retrieve.

Step 1: Analyze user request

→ "Analyze customer logs and identify why API request #909 failed."

Step 2: Tool selection (Sub-Query)

→ Agent chooses to call `QueryVectorDB("API_909")`

Step 3: Reflection loop

→ Looks at output: "Logs mention server connection error 504 on node B."

→ Decision: "Now query DB for Server Node B hardware logs."

Ashish Jangra

linkedin.com/in/ashish-jangra



TAXONOMY COMPARISON

The Paradigm Matrix

Architecture	Dynamic Loops	Best Use Case	Cost Profile
Naive RAG	No (Linear)	Small, simple FAQs	Very Low
Advanced RAG	Limited	Technical Docs	Low-Medium
Graph RAG	Entity-driven	Relational mapping	High (Graph Build)
Agentic RAG	Fully Loop-based	Multi-step audits	Very High

Engineering Choice: Start with **Advanced RAG**. Scale to **Graph RAG** if queries cross multiple tables, or **Agentic RAG** for open-ended troubleshooting work.



Types of RAG

CONCEPT UNLOCKED



Naive RAG is simple, linear, but highly fragile



Advanced RAG adds pre/post layers like Re-ranking



Graph RAG maps multi-hop network connections



Agentic RAG drives autonomous multi-step checks

CONNECT WITH ME



LinkedIn



GitHub



Kaggle



Instagram

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)

